

DAY 8

1. Command Substitution

What is Command Substitution?

Command substitution allows you to:

Execute a command and substitute its output into another command or store that output in a variable.

The syntax is:

```
$(command)
```

For example:

```
$(pwd)
```

The shell executes:

```
pwd
```

and replaces `$(pwd)` with the command's output.

Basic Example

```
current_dir=$(pwd)
```

Here:

```
pwd
↓
executes command
↓
produces output
↓
output is stored in current_dir
```

Now:

```
echo "${current_dir}"
```

might produce:

```
/home/sourya/Documents
```

So:

```
current_dir=$(pwd)
```

means:

| Run `pwd` and store its output in `current_dir`.

2. Why Is Command Substitution Useful?

Command substitution is extremely useful in shell scripting because the output of one command can become the **input/data for another part of the script**.

For example:

```
hostname=$(hostname)
```

Then:

```
echo "This machine is ${hostname}"
```

Another example:

```
file_count=$(find . -type f | wc -l)
```

Now the number of files can be used elsewhere:

```
echo "Number of files: ${file_count}"
```

This is one of the foundations of **automation in Bash**.

3. Command Substitution vs Parameter Expansion

These two concepts are related but different.

Parameter Expansion

Works with the **value of a parameter/variable**.

```
name="Sourya"

echo "${name}"
echo "${name^^}"
```

You're manipulating data that already exists in a variable.

Command Substitution

Works with the **output of a command**.

```
hostname=$(hostname)
```

You're executing a command and capturing its output.

Easy way to remember

```
Parameter Expansion
    ↓
Manipulate a value

Command Substitution
    ↓
Execute command → capture output
```

4. Command Substitution Directly in a Command

You don't always have to store the output in a variable.

You can use it directly:

```
echo "Today is $(date)"
```

The shell executes:

```
date
```

and inserts its output into the `echo` command.

Conceptually:

```
echo "Today is $(date)"
    ↓
  execute date
    ↓
  get date/time
    ↓
  substitute output
    ↓
  echo final text
```

5. Nested Command Substitution

Command substitution can also be nested.

For example:

```
echo "User: $(whoami)"
```

The output of `whoami` becomes part of the `echo` command.

In more complicated scripts, command substitutions can be combined with variables and other commands.

6. Old Command Substitution Syntax

Historically, command substitution could also be written using backticks:

```
`command`
```

Example:

```
current_dir=`pwd`
```

This works in many shells, but the preferred modern syntax is:

```
current_dir=$(pwd)
```

Why prefer `$( )` ?

Because `$( )` :

- Is easier to read
- Is easier to nest
- Is generally clearer in scripts

Therefore, prefer:

```
$(command)
```

over:

```
`command`
```

7. Practical Security/SOC Examples

Command substitution appears frequently in security scripts.

For example:

```
username=$(whoami)
hostname=$(hostname)
ip=$(hostname -I)
```

Now the script can use these values:

```
echo "User: ${username}"  
echo "Host: ${hostname}"  
echo "IP: ${ip}"
```

A security script could gather system information automatically:

```
Command  
  ↓  
whoami  
  ↓  
Command substitution  
  ↓  
username variable  
  ↓  
Use in report
```

This is why understanding command substitution is important for **SOC automation, Bash scripts, and incident-response tooling**.

8. Arithmetic Expansion

Bash provides **arithmetic expansion** for performing mathematical calculations.

The syntax is:

```
$(( expression ))
```

For example:

```
echo $((5 + 3))
```

Output:

```
8
```

The shell evaluates the mathematical expression and replaces `$((...))` with the result.

9. Basic Arithmetic Operators

The major operators are:

Operator	Meaning	Example	Result
+	Addition	<code>5 + 3</code>	8
-	Subtraction	<code>5 - 3</code>	2
*	Multiplication	<code>5 * 3</code>	15
/	Division	<code>10 / 2</code>	5
%	Modulo/remainder	<code>10 % 3</code>	1
**	Exponentiation	<code>2 ** 3</code>	8

10. Addition

```
echo $((10 + 5))
```

Output:

```
15
```

You can also use variables:

```
a=10
b=5

result=$((a + b))

echo "${result}"
```

Output:

```
15
```

Notice that inside arithmetic expansion, you generally don't need `$` before variable names:

```
=$((a + b))
```

rather than:

```
=$(( $a + $b ))
```

Both may work in many Bash contexts, but the first is the normal style.

11. Subtraction

```
a=20
b=8

result=$((a - b))

echo "${result}"
```

Output:

```
12
```

12. Multiplication

Use `*` :

```
a=6
b=7

result=$((a * b))
```

```
echo "${result}"
```

Output:

```
42
```

13. Division

```
a=20  
b=4  
  
result=$((a / b))  
  
echo "${result}"
```

Output:

```
5
```

Important: Integer Division

Bash's normal arithmetic expansion works with **integers**.

For example:

```
echo $((5 / 2))
```

produces:

```
2
```

not:

```
2.5
```

The fractional part is discarded.

This limitation is important.

14. Order of Operations

Bash follows normal mathematical operator precedence.

For example:

```
echo $((2 + 3 * 4))
```

Multiplication happens first:

```
3 × 4 = 12  
2 + 12 = 14
```

Result:

```
14
```

It does **not** calculate:

```
(2 + 3) × 4 = 20
```

15. Using Parentheses

You can use parentheses to explicitly control the order of calculation.

Example:

```
echo $(( (2 + 3) * 4 ))
```

First:

```
2 + 3 = 5
```

Then:

```
5 × 4 = 20
```

Output:

```
20
```

Remember

Without parentheses:

```
2 + 3 * 4 = 14
```

With parentheses:

```
(2 + 3) * 4 = 20
```

16. Exponentiation

Bash supports exponentiation using:

```
**
```

Example:

```
echo $((2 ** 3))
```

Output:

```
8
```

Because:

```
23 = 2 × 2 × 2 = 8
```

Another example:

```
echo $((5 ** 2))
```

Output:

17. Modulo Operator %

The modulo operator returns the **remainder after division**.

Syntax:

```
a % b
```

Example:

```
echo $((10 % 3))
```

Output:

```
1
```

Because:

```
10 ÷ 3 = 3 remainder 1
```

Another example:

```
echo $((20 % 5))
```

Output:

```
0
```

because 20 divides evenly by 5.

18. Why Modulo Is Useful

Modulo is extremely useful in scripting.

Check whether a number is even

```
if (( number % 2 == 0 )); then
    echo "Even"
fi
```

Because:

```
even number % 2 = 0
```

Check odd numbers

```
odd number % 2 = 1
```

Modulo is also useful for:

- Counters
- Repeating tasks
- Scheduling
- Loop logic
- File batching
- Checking divisibility

19. Arithmetic Expansion with Variables

Example:

```
x=15
y=4

sum=$((x + y))
difference=$((x - y))
product=$((x * y))
quotient=$((x / y))
remainder=$((x % y))

echo "Sum: ${sum}"
echo "Difference: ${difference}"
echo "Product: ${product}"
```

```
echo "Quotient: ${quotient}"  
echo "Remainder: ${remainder}"
```

Output:

```
Sum: 19  
Difference: 11  
Product: 60  
Quotient: 3  
Remainder: 3
```

Notice again:

```
$(x + y)
```

is **arithmetic expansion**, while:

```
${x}
```

is **parameter expansion**.

20. The Problem with Decimal Numbers

Standard Bash arithmetic expansion is designed primarily for **integer arithmetic**.

For example:

```
echo $(10 / 3)
```

produces:

```
3
```

But mathematically:

```
10 / 3 = 3.333333...
```

Bash doesn't directly provide normal floating-point arithmetic through `$(( ))`.

So for decimal calculations, we commonly use:

```
bc
```

21. What is `bc`?

`bc` stands for **Basic Calculator**.

It is a command-line calculator available on many Unix/Linux systems.

It supports calculations involving:

- Integers
- Decimals
- Arithmetic expressions
- Precision control
- More advanced mathematical operations

22. Using a Pipe with `bc`

We can send an expression to `bc` using a pipe:

```
echo "10 / 3" | bc
```

The flow is:

```
echo "10 / 3"  
  ↓  
 pipe |  
  ↓  
  bc  
  ↓  
calculate  
  ↓  
result
```

However, by default:

```
echo "10 / 3" | bc
```

may return:

```
3
```

because `bc` also defaults to zero decimal places for division unless precision is specified.

23. The `scale` Variable

`bc` provides an internal variable called:

```
scale
```

It controls the number of decimal places used in calculations such as division.

For example:

```
echo "scale=2; 10 / 3" | bc
```

Output:

```
3.33
```

Here:

```
scale=2
```

means:

| Keep 2 digits after the decimal point.

24. More `scale` Examples

One decimal place

```
echo "scale=1; 10 / 3" | bc
```

Result:

```
3.3
```

Four decimal places

```
echo "scale=4; 10 / 3" | bc
```

Result:

```
3.3333
```

Six decimal places

```
echo "scale=6; 10 / 3" | bc
```

Result:

```
3.333333
```

25. Decimal Addition

`bc` can also handle decimal values.

```
echo "10.5 + 2.25" | bc
```

Result:

```
12.75
```

26. Decimal Multiplication

```
echo "10.5 * 2.5" | bc
```

Result:

```
26.25
```

27. Decimal Division

```
echo "scale=3; 10 / 3" | bc
```

Result:

```
3.333
```

This is particularly useful when calculating:

- Percentages
 - Average values
 - Rates
 - Network statistics
 - Performance measurements
 - Time calculations
-

28. Storing `bc` Output in a Variable

Because `bc` produces output, we can combine it with **command substitution**.

For example:

```
result=$(echo "scale=2; 10 / 3" | bc)
```

Now:

```
echo "${result}"
```

Output:

```
3.33
```

This combines three concepts:

```
Command Substitution
      ↓
$( ... )
      ↓
Execute command pipeline
      ↓
echo "scale=2; 10 / 3" | bc
      ↓
Capture output
      ↓
Store in variable
```

This is a very important real-world Bash pattern.

29. Command Substitution + Arithmetic: Complete Example

Suppose you want to calculate an average.

```
total=250
count=4

average=$(echo "scale=2; ${total} / ${count}" | bc)

echo "Average: ${average}"
```

Output:

```
Average: 62.50
```

Here you're using:

1. Variables
2. Parameter expansion
3. Command substitution
4. Pipe
5. `bc`
6. `scale`

All together.
